

# CASOS REALES DE FALLAS EN EL SOFTWARE



- Fallas comunes
- Sus consecuencias
- Ejemplos de casos de estudio
- ¿Cómo prevenirlas?  
Mediante prácticas de seguridad y pruebas.

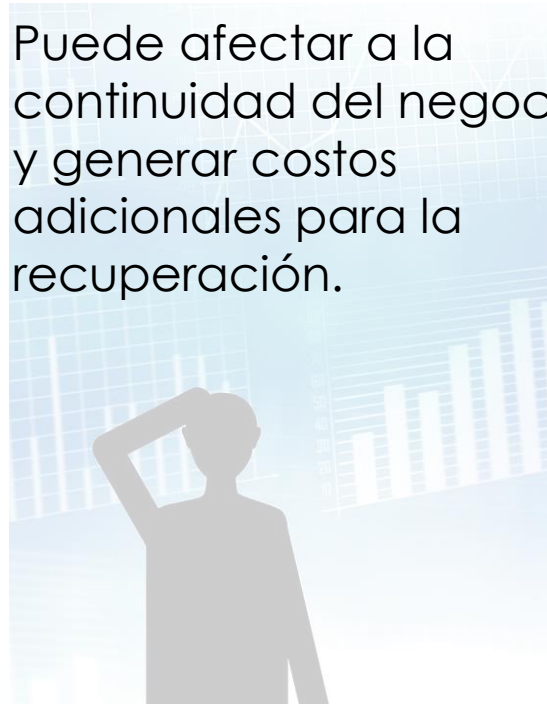
# ***Errores comunes en el desarrollo de Software***

- **Falta de comunicación y colaboración entre los miembros del equipo de desarrollo.**
- Falta de pruebas exhaustivas, lo que puede llevar a la detección tardía de errores.
- **Uso de lenguajes de programación o herramientas inadecuadas.**
- Mala gestión de los requisitos del proyecto, lo que puede llevar a la implementación de funcionalidades incompletas o incorrectas.
- **Falta de experiencia y capacitación del equipo de desarrollo.**
- Presión por completar el proyecto en un plazo corto, lo que puede llevar a la toma de decisiones apresuradas y a la omisión de pasos importantes en el proceso de desarrollo.

# Estas consecuencias pueden incluir:

## Pérdida de datos

Puede afectar a la continuidad del negocio y generar costos adicionales para la recuperación.



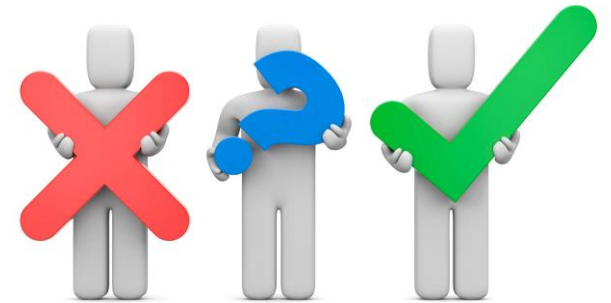
## Pérdida de tiempo y productividad

Eficiencia de la empresa y reducir los beneficios.



## Pérdida de confianza de los clientes

Afectar a la reputación de la empresa y generar pérdidas de ingresos.



# Ejemplos de casos de estudio de fallas

## Ariane 5

Desarrollado por la agencia espacial europea, sufrió una falla catastrófica en su vuelo inaugural en 1996.

La falla se produjo debido a un error de software que provocó la pérdida de control del cohete y su destrucción. El error se produjo cuando se intentó convertir un valor de 64 bits a un valor de 16 bits, lo que provocó un desbordamiento y una interpretación incorrecta de los datos. El error se debió a una mala gestión de los requisitos del proyecto y a la falta de pruebas exhaustivas del software.



# Ejemplos de casos de estudio de fallas

## Therac-25

- El Therac-25, una máquina de radioterapia desarrollada por la empresa canadiense Atomic Energy of Canada Limited, fue responsable de varios accidentes que causaron lesiones graves e incluso la muerte de pacientes. La falla se produjo debido a un error de software que permitía que la máquina entregara **dosis de radiación extremadamente altas**, superando los límites seguros. El error se debió a una **mala gestión de la seguridad del software**, a la falta de pruebas exhaustivas, y a la falta de un **sistema de seguridad** adecuado para evitar errores humanos.



# Prácticas de seguridad en el desarrollo de software



**Análisis de riesgos:** identificar los posibles riesgos de seguridad y desarrollar estrategias para mitigarlos.



**Desarrollo seguro:** incorporar la seguridad en el proceso de desarrollo, incluyendo la codificación segura, la gestión de vulnerabilidades y la revisión del código.



**Pruebas de seguridad:** realizar pruebas exhaustivas para detectar vulnerabilidades y errores de seguridad.

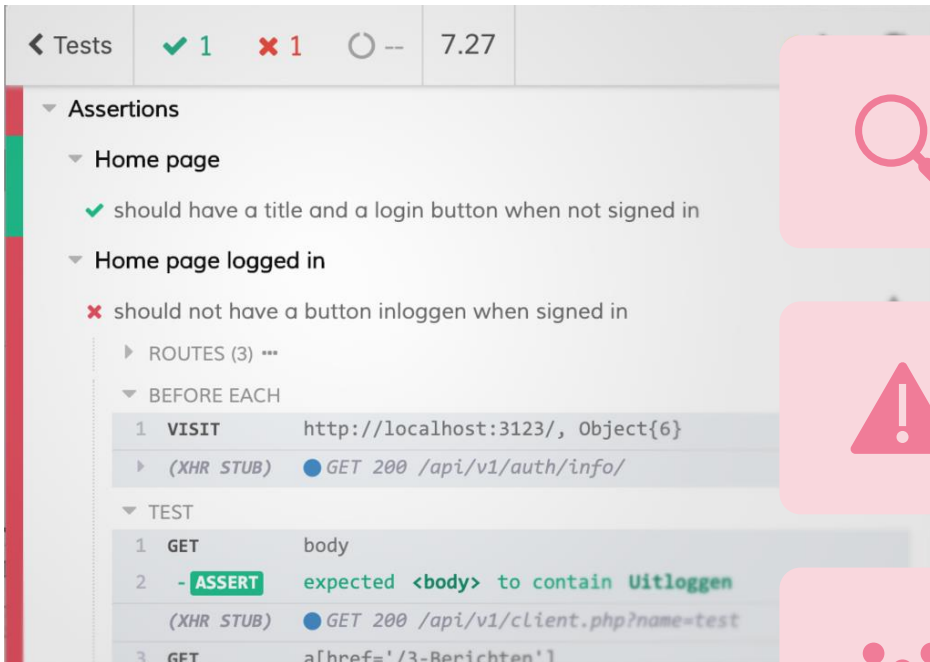


**Gestión de vulnerabilidades:** gestionar las vulnerabilidades descubiertas, incluyendo su corrección y la actualización del software.



**Capacitación del equipo:** capacitar al equipo de desarrollo en seguridad del software, incluyendo las mejores prácticas y las herramientas disponibles.

Para garantizar la seguridad del software es fundamental implementar prácticas de seguridad durante **todo el ciclo de vida del desarrollo**.



A large, irregular pink brushstroke shape on the left side of the slide, serving as a background for the title.

## ***Técnicas de pruebas y validación de software***

Las pruebas de software son un proceso fundamental para garantizar la calidad y la seguridad del software.

Existen diversas técnicas de pruebas que se pueden aplicar, incluyendo:





## *Técnicas de pruebas*

**Pruebas unitarias:** verificar el correcto funcionamiento de las unidades individuales del software.

**Pruebas de integración:** verificar el correcto funcionamiento de las unidades integradas entre sí.

**Pruebas de sistema:** verificar el correcto funcionamiento del software completo en un entorno real.

**Pruebas de aceptación:** verificar si el software cumple con los requisitos del cliente.





## *Técnicas de pruebas*

### **Pruebas de rendimiento:**

evaluar la eficiencia y la estabilidad del software bajo cargas altas.

### **Pruebas de seguridad:**

identificar vulnerabilidades y errores de seguridad en el software.

La validación de software implica verificar que el software cumple con los requisitos especificados y que es adecuado para su propósito.

# Recomendaciones para evitar fallas de software



Establecer una cultura de seguridad en el equipo de desarrollo, donde la seguridad sea una prioridad desde el inicio del proyecto.



Implementar un proceso de desarrollo seguro que incluya la identificación de riesgos, la codificación segura, la gestión de vulnerabilidades, la revisión del código y las pruebas de seguridad.



Utilizar herramientas de análisis de código y de pruebas de seguridad para identificar y corregir errores de seguridad en el software.



Mantener actualizado el software, corrigiendo las vulnerabilidades descubiertas y actualizando los componentes del software.



Capacitar al equipo de desarrollo en seguridad del software, incluyendo las mejores prácticas y las herramientas disponibles.

# Recomendaciones

- Las fallas en el software pueden tener consecuencias graves y, por lo tanto, es fundamental implementar prácticas de seguridad durante todo el ciclo de vida del desarrollo. La seguridad debe ser una prioridad desde el inicio del proyecto, y es fundamental contar con un equipo de desarrollo que esté capacitado y comprometido con la seguridad del software.
- Las técnicas de pruebas y validación de software son esenciales para garantizar la calidad y la seguridad del software. Es fundamental aplicar una combinación de técnicas de pruebas para garantizar que el software esté libre de errores y vulnerabilidades.
- Para evitar fallas de software, se recomienda establecer una cultura de seguridad en el equipo de desarrollo, implementar un proceso de desarrollo seguro, utilizar herramientas de análisis de código y de pruebas de seguridad, mantener actualizado el software y capacitar al equipo de desarrollo en seguridad del software.

# Actividad Práctica: Simulación de Login Seguro (Web)

**Objetivo:** implementar un sistema que utilice **hashing** y **salting** para almacenar las contraseñas de los usuarios, y que verifique las credenciales durante el proceso de autenticación. No es necesario implementar la persistencia de datos; el sistema puede utilizar un almacenamiento en memoria para simplificar el desarrollo.

# Requerimientos

- La simulación debe incluir una **interfaz web** sencilla que permita a los usuarios **registrarse y autenticarse**. Durante el registro, el sistema debe **generar una sal aleatoria para cada usuario, concatenarla con la contraseña (opcional)** proporcionada y aplicar un algoritmo de hashing robusto (como SHA-256) para obtener el hash de la contraseña. La sal y el hash deben ser almacenados en memoria.

# Funcionamiento

- Durante la autenticación, el sistema debe recuperar la sal correspondiente al usuario, concatenarla con la contraseña proporcionada y aplicar el mismo algoritmo de hashing para obtener un nuevo hash. Este nuevo hash debe ser comparado con el hash almacenado en memoria. Si los hashes coinciden, el usuario es autenticado con éxito; de lo contrario, el acceso es denegado.

# Funcionamiento

- Además, se recomienda implementar medidas de seguridad adicionales, como la limitación del número de intentos fallidos de autenticación para prevenir ataques de fuerza bruta. El sistema también debe mostrar mensajes de error genéricos para evitar revelar información sobre la existencia de cuentas o la validez de las contraseñas.